



Distributed Systems Client-Server Computing TCP Server

Ref: Unix Network Programming, The Socket Networking API by W Richard Stevens



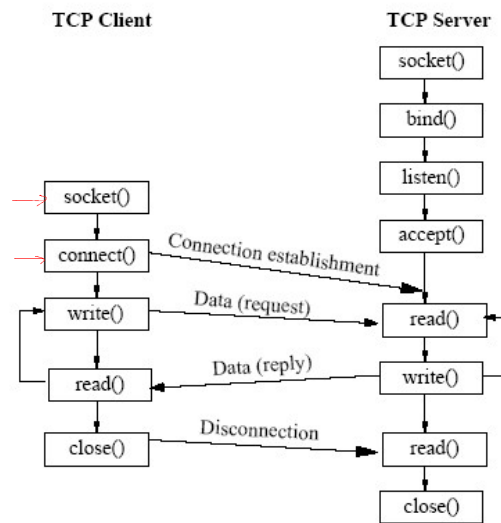
Recap



- TCP
- TCP – Design issues
- BSD Socket API
- TCP Client / Server - Function calls
- Socket Address Structure
- Socket function

```
int socket(int family, int type, int protocol);
```

Socket system calls for connection-oriented protocol



IIT ROORKEE

```

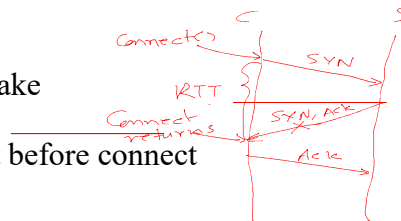
int
main(int argc, char **argv)
{
    int    sockfd, fd;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    printf("%d\n", sockfd);
    fd = open("data", O_RDONLY);
    printf("%d\n", fd);

    for(;;);
    exit(0);
}
  
```

```
int connect(int sockfd, struct sockaddr *servaddr, int addrlen);
```

- ✓ Initiates TCP three way handshake
- ✓ Client does not have to call bind before connect
- ✓ No response from server
- ✓ Server sends RST (hard error) – reasons?
- ✓ ICMP destination unreachable (soft error)



```
int bind(int sockfd, const struct sockaddr *myaddr, int addrlen);
```

- Server registers their well known address with the system
 - connection-oriented TCP
 - connectionless UDP
- TCP server bind their well known port when they start
- TCP client?
- Client can also register a specific address for itself

→ TCP server calls bind
 " " does not call bind
 " " also bind but does not specify IP
 does not specify port

```
int bind(int sockfd, const struct sockaddr *myaddr, int addrlen);
```

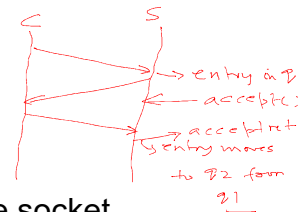
Port

- If TCP server or client does not call bind
 - kernel chooses an ephemeral port
 - **when?** *client → connect TCP server listen*
- Process calls bind and specifies port (0 / nonzero)

IP Address

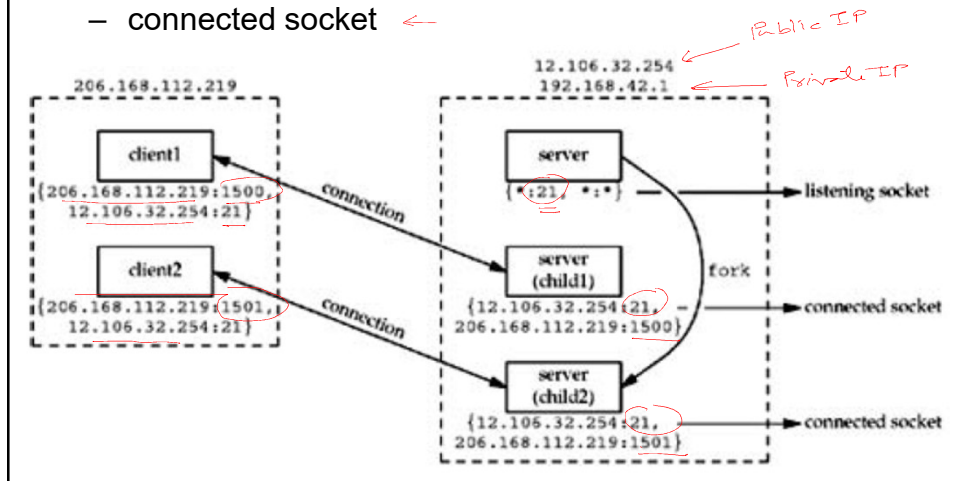
- A process can bind a specific IP address to its socket
- If bind is not called?
 - o Kernel chooses the IP address (when?) *Server — when first SYN arrives*
- Process calls bind and specifies
 - o IP address (wildcard / IP address)

```
int listen(int sockfd, int backlog);
```

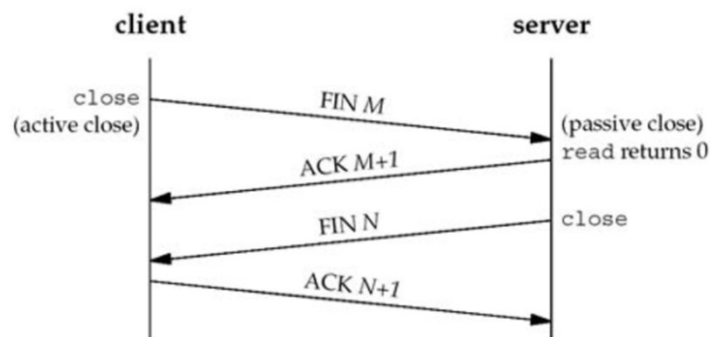


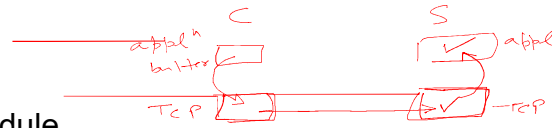
- Called only by a TCP server
- by default a socket is created as an active socket
- call to listen converts it to a passive socket
- kernel maintains two queues for each listening socket
 - Incomplete connection queue *p1*
 - Completed connection queue *p2*
- Fudge factor (Berkely derived implementations)
- If queues are full when a client SYN arrives?

- `int accept(int sockfd, struct sockaddr *peer, int *addrlen);`
 - First entry in the completed queue is returned
 - listening socket
 - connected socket ←



- `int close(int sockfd);`
 - mark the socket as closed
 - No read / write possible on this descriptor
 - TCP sends remaining data in buffer
 - **and sends FIN?**
 - Only when descriptor reference count reaches zero





• Can we tell the TCP module

- Wait till **application knows** that TCP has delivered all data in buffer to the peer process
- ○ to **discard the buffer** and send FIN immediately?
- ○ to send FIN even if **other processes have not closed the descriptor**?